

L1 Cache Sensitivity Analysis of Matrix Multiplication and AES Encryption on X86 Architecture using Gem5 Simulator

Kush Mehta **2**3BEC096
Mitanshu Patel **2**3BEC108

Course: 3EC503CC24 - Computer Architecture

March 6, 2026

Abstract

This report presents a comprehensive microarchitectural analysis of Level 1 (L1) Data Cache dynamics using the Gem5 simulator. We evaluated the impact of cache capacity, workload memory access patterns (Matrix Multiplication vs. AES Cryptography), and cache associativity on an X86-based `TimingSimpleCPU`. The results demonstrate that increasing capacity from 4kB to 64kB reduces capacity misses from 5.06% to 0.04% for linear algebraic workloads, while cryptographic payloads remain indifferent to capacity scaling. Furthermore, an evaluation of mapping schemes at a constrained 16kB capacity revealed that increasing associativity from Direct-Mapped to 8-Way Set-Associative effectively mitigates conflict misses, improving IPC by 10.4%.

Keywords: Gem5 Simulator, Cache Memory, IPC, Miss Rate, Associativity, AES, Matrix Multiplication, X86 Architecture.

1 Literature Survey

Modern processors are critically bottlenecked by the ”Memory Wall,” requiring sophisticated cache hierarchies to maintain data throughput.

- **Miss Classifications:** Cache inefficiencies are broadly categorized into the ”Three C’s”: Compulsory, Capacity, and Conflict misses. Architectural design aims to balance silicon area and latency to address these bottlenecks [1].
- **Workload Diversity:** Programs exhibit varying degrees of temporal and spatial locality. Linear algebraic kernels heavily utilize spatial locality, while cryptographic algorithms (like AES S-Box lookups) frequently present non-linear, scattered access patterns [2].
- **Simulation Tools:** The Gem5 simulator provides cycle-accurate models of CPU pipelines and memory subsystems, enabling precise evaluation of hardware-software co-design without physical fabrication [3].

2 Limitations of Available Technology

In resource-constrained environments such as embedded IoT devices and UAV flight controllers, hardware cannot be dynamically resized.

- **Thrashing and Conflicts:** Fixed, Direct-Mapped caches suffer from severe conflict misses when executing matrix-heavy operations (e.g., sensor fusion), leading to pipeline stalls.
- **Power Constraints:** While highly associative caches (e.g., 8-Way) eliminate conflict misses, they require complex, power-hungry multiplexing logic, making them unviable for ultra-low-power edge nodes.

3 Proposed Methodology

The experiments were conducted using Gem5 (Version 25.1) simulating an X86 `TimingSimpleCPU`.

1. **Capacity Experiment:** Executed a 100x100 Matrix Multiplication C-benchmark (working set $\approx$ 40kB). The L1 Data cache was varied across 4kB, 32kB, and 64kB (Default Associativity).
2. **Workload Sensitivity:** Executed an AES SubBytes benchmark (150kB payload) across the same capacities to contrast memory access behavior.
3. **Associativity Experiment:** The matrix benchmark was executed with the L1 Data cache locked at 16kB. The mapping scheme was varied across 1-Way (Direct-Mapped), 2-Way, and 8-Way Set-Associativity.

4 Flow Chart

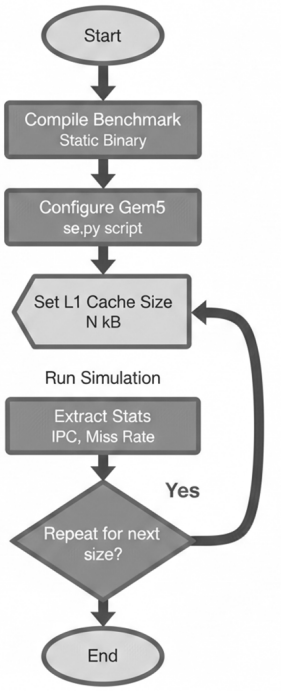


Figure 1: Simulation Flowchart

The process flow involves compiling the benchmark into a static binary, configuring the `se.py` simulation script, and iteratively running the simulation while modifying the L1 data cache parameters.

5 Simulation Results

5.1 Experiment 1: L1 Data Cache Size (Capacity)

L1 Data Cache Size	IPC (Speed)	Miss Rate (Efficiency)
4kB	0.277	5.06%
32kB	0.297	1.82%
64kB	0.310	0.04%

Table 1: Impact of Capacity on Matrix Multiplication

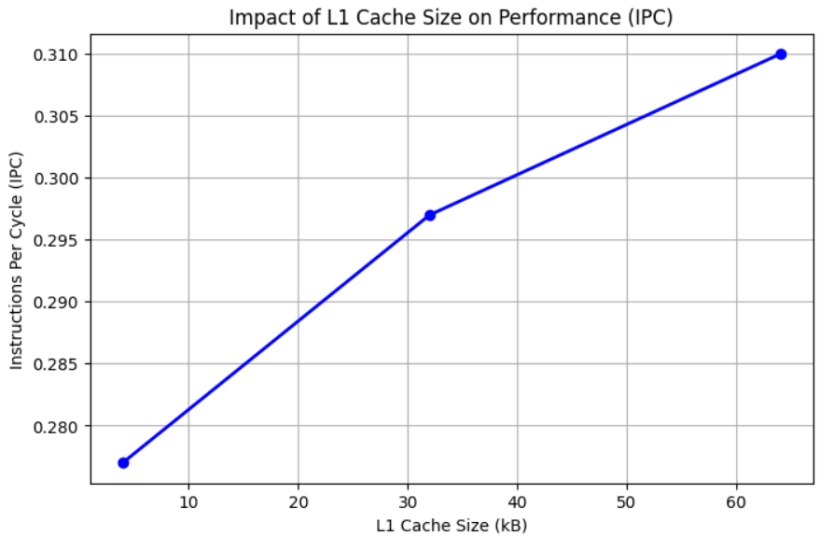


Figure 2: L1 Cache Size vs. IPC

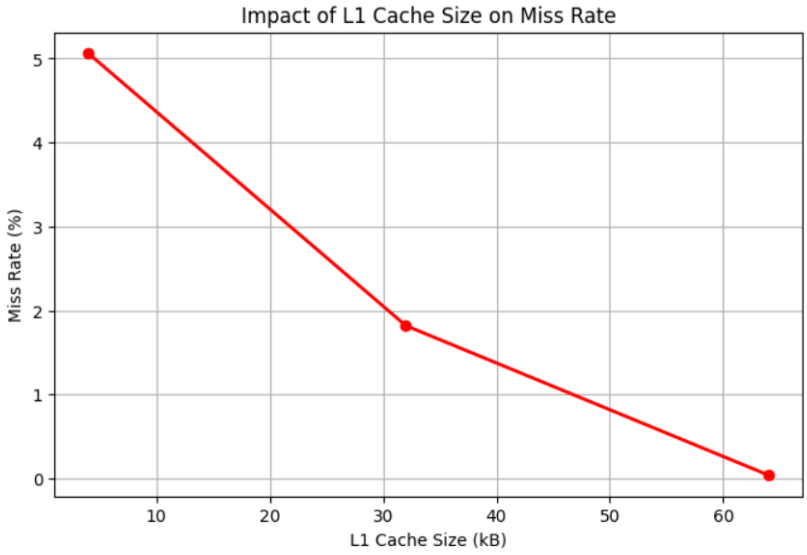


Figure 3: L1 Cache Size vs. Miss Rate

5.2 Experiment 2: Workload Sensitivity (AES Cryptography)

L1 Data Cache Size	IPC (Speed)	Miss Rate (Efficiency)
4kB	0.259	0.14%
32kB	0.259	0.14%
64kB	0.263	0.14%

Table 2: Impact of Capacity on AES Encryption

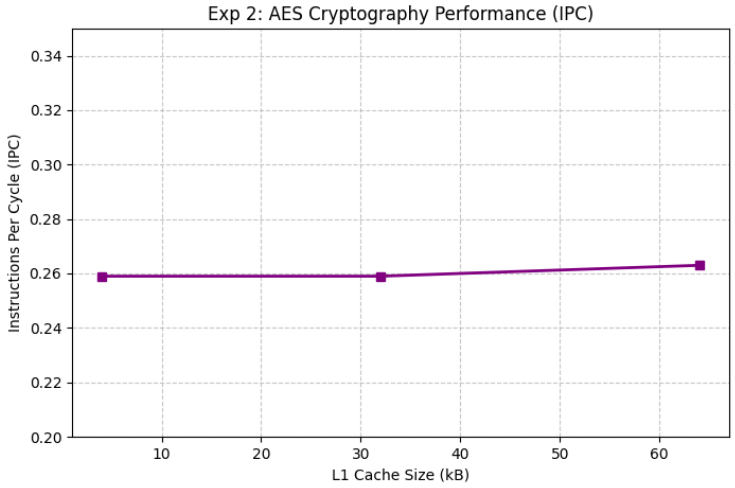


Figure 4: L1 Cache Size vs. IPC

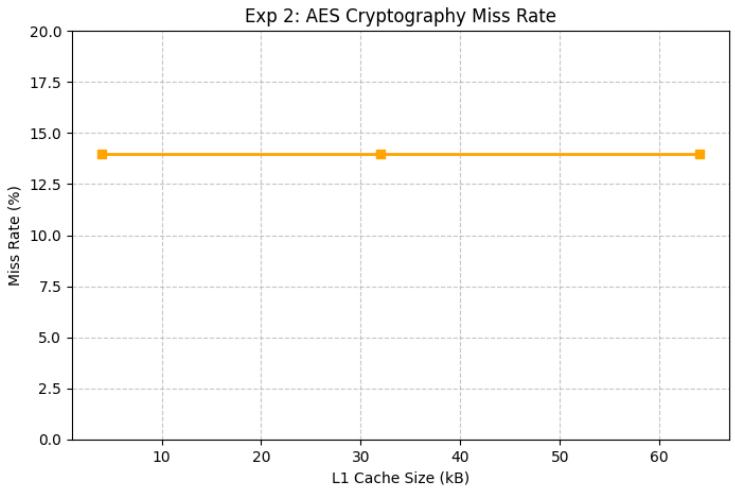


Figure 5: L1 Cache Size vs. Miss Rate

5.3 Experiment 3: Mapping Schemes (Associativity at 16kB)

Associativity (16kB Cache)	IPC (Speed)	Miss Rate (Efficiency)
1-Way (Direct Mapped)	0.279	4.74%
2-Way Set Associative	0.296	2.07%
8-Way Set Associative	0.308	0.34%

Table 3: Impact of Cache Associativity on Matrix Multiplication

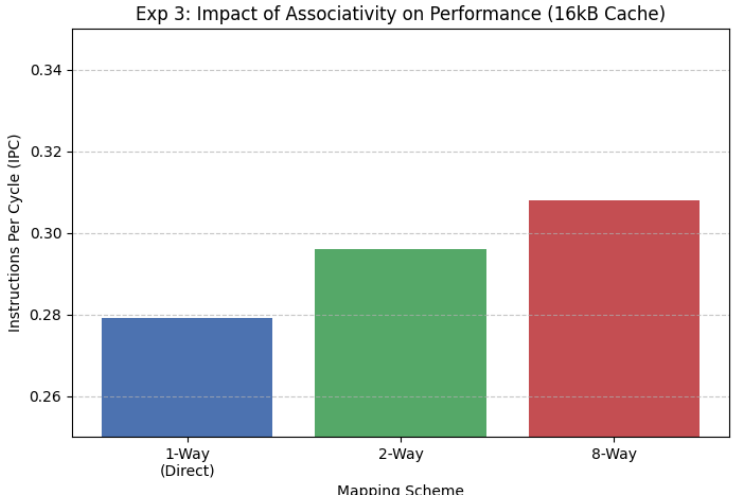


Figure 6: L1 Cache Size vs. IPC

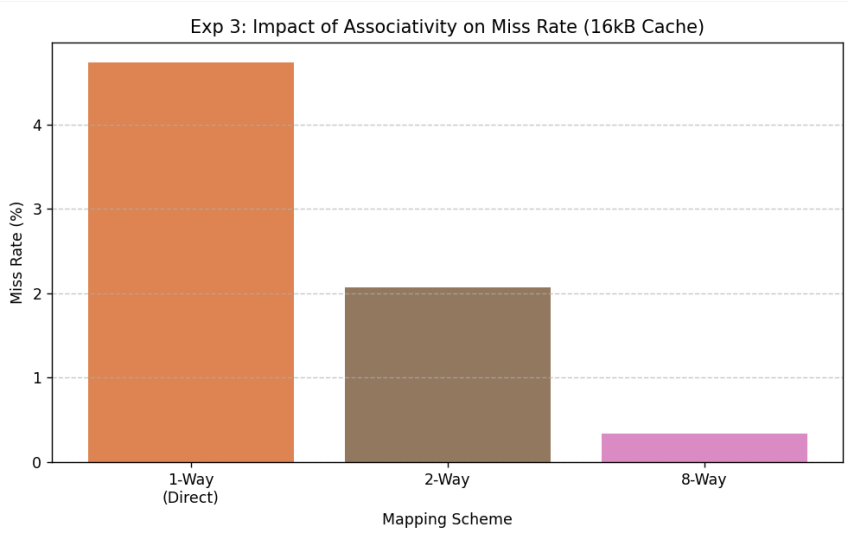


Figure 7: L1 Cache Size vs. Miss Rate

6 Analysis of Obtained Results

1. Mitigating Capacity Misses: In Experiment 1, the 4kB cache thrashing yielded a 5.06% miss rate because it was smaller than the 40kB working set. Expanding the cache to 64kB captured the entire set, dropping misses to a negligible 0.04% (purely compulsory misses).

2. Algorithmic Sensitivity: Experiment 2 revealed that lightweight cryptographic workloads behave differently. Because the AES S-Box is only 256 bytes, it fits entirely within the smallest 4kB cache. The scattered, non-linear lookups and linear payload streaming resulted in a static 0.14% miss rate, proving that simply adding cache memory does not solve pipeline execution bottlenecks for all algorithms.

3. Mitigating Conflict Misses: In Experiment 3, constraining the cache to 16kB forced contention. Under a Direct-Mapped (1-Way) scheme, the 4.74% miss rate heavily restricted the IPC. Transitioning to a 2-Way set-associative layout halved the miss rate to 2.07%, confirming the elimination of severe conflict misses. The 8-way scheme provided diminishing returns (0.34% misses), demonstrating the classic architectural trade-off between associativity benefits and hardware complexity.

7 Conclusion

This study validates that cache optimization requires a holistic view of the targeted workload. While data-heavy linear algorithms benefit exponentially from larger capacities and flexible associativity mappings to prevent conflict and capacity misses, specialized edge workloads like cryptography are bottlenecked elsewhere. Optimizing hardware for UAVs or IoT devices requires tailoring the cache hierarchy—such as utilizing highly-associative, smaller caches for dense matrices—rather than relying on brute-force capacity expansion.

8 References

1. Hennessy, J. L., & Patterson, D. A. (2003). *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann.

2. Binkert, N., et al. (2011). "The Gem5 simulator". *SIGARCH Computer Architecture News*.

3. 3EC503CC24 Course Assignment Topic List (2025).

Appendix: Code

A. Benchmark Code (matrix_mul.c)

```c
#include <stdio.h>
#define N 100

void multiply(int mat1[][N], int mat2[][N], int res[][N]) {
    int i, j, k;
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            res[i][j] = 0;
            for (k = 0; k < N; k++)
                res[i][j] += mat1[i][k] * mat2[k][j];
        }
    }
}

int main() {
    int mat1[N][N], mat2[N][N], res[N][N];
    // Initialization omitted for brevity
    // In actual code, matrices are initialized with dummy data
    multiply(mat1, mat2, res);
    return 0;
}
```

A. Benchmark Code (aes_benchmark.c)

```c
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>

#define PAYLOAD_SIZE 150000 // 150kB payload (designed to fit in 256kB
   L2, but miss in 4kB/32kB L1)
#define ITERATIONS 50       // Repeat to simulate continuous IoT data
   streaming

// Standard AES S-Box (256 bytes)
static const uint8_t sbox[256] = {
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0
       x2b, 0xfe, 0xd7, 0xab, 0x76,
    0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0
       xaf, 0x9c, 0xa4, 0x72, 0xc0,
    0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0
       xf1, 0x71, 0xd8, 0x31, 0x15,
```

```
    0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0
        xe2, 0xeb, 0x27, 0xb2, 0x75,
    0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0
        xb3, 0x29, 0xe3, 0x2f, 0x84,
    0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0
        x39, 0x4a, 0x4c, 0x58, 0xcf,
    0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0
        x7f, 0x50, 0x3c, 0x9f, 0xa8,
    0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0
        x21, 0x10, 0xff, 0xf3, 0xd2,
    0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0
        x3d, 0x64, 0x5d, 0x19, 0x73,
    0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0
        x14, 0xde, 0x5e, 0x0b, 0xdb,
    0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0
        x62, 0x91, 0x95, 0xe4, 0x79,
    0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0
        xea, 0x65, 0x7a, 0xae, 0x08,
    0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0
        x1f, 0x4b, 0xbd, 0x8b, 0x8a,
    0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0
        xb9, 0x86, 0xc1, 0x1d, 0x9e,
    0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0
        xe9, 0xce, 0x55, 0x28, 0xdf,
    0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0
        x0f, 0xb0, 0x54, 0xbb, 0x16
};

// Simulate the SubBytes step of AES
void aes_sub_bytes(uint8_t *data, int length) {
    for (int i = 0; i < length; i++) {
        // The S-Box lookup: This is a scattered, non-linear memory
            access
        data[i] = sbox[data[i]];
    }
}

int main() {
    // Allocate a large payload of "sensor data"
    uint8_t *payload = (uint8_t *)malloc(PAYLOAD_SIZE);

    // Initialize with dummy data
    for (int i = 0; i < PAYLOAD_SIZE; i++) {
        payload[i] = i % 256;
    }

    // Run the encryption substitution heavily to stress the cache
```

```
    for (int iter = 0; iter < ITERATIONS; iter++) {
        aes_sub_bytes(payload, PAYLOAD_SIZE);
    }

    // Prevent compiler from optimizing the loop away
    printf("Final byte check: %x\n", payload[PAYLOAD_SIZE - 1]);

    free(payload);
    return 0;
}
}
```

B. Gem5 Simulation Commands

```
# Compilation
gcc -static matrix_mul.c -o matrix_mul

# Execution (Example for 64kB)
# Note: Path to se.py may vary based on Gem5 version
build/X86/gem5.opt configs/deprecated/example/se.py \
  --cmd=../my_cache_project/matrix_mul \
  --cpu-type=TimingSimpleCPU \
  --caches --l2cache \
  --l1d_size=4kB --l1i_size=4kB --l2_size=128kB
```